

Activity 4: LeetCode Practice & Portfolio Integration

GitHub Repository: <https://github.com/Yagami-akshay/leetcode-solutions.git>

Student Name: Akshay N | SRN: R25EF021 | Class & Section: CSE-B / 3rd Sem

Yagami-akshay / leetcode-solutions

Signed in as Yagami-akshay (Akshay N - R25EF021)

arrays-strings

Add Arrays & Strings solutions: Two Sum, Reverse String, and Valid Anagram

basic-algorithms

Add Basic Algorithms: Binary Search and Move Zeroes

linked-lists

Add Linked Lists (Bonus): Reverse Linked List

stacks

Add Stacks: Valid Parentheses with local test suite

.gitignore

Initialize leetcode-solutions repository

DELIVERABLES.md

Add DELIVERABLES.md fulfilling Section 10 deliverables checklist

PROGRESS.md

Update student details: Class & Section CSE-B 3rd sem

README.md

Update student details: Class & Section CSE-B 3rd sem

README.md

LeetCode Solutions & Practice Log

Student Name: Akshay N

SRN: R25EF021

Class & Section: CSE-B (3rd Semester)

Course: Portfolio Building for Engineering Students — GitHub-Integrated Edition (B25GE0101)

Personal LeetCode practice log — part of B25GE0101 portfolio

Table of Contents

• Arrays & Strings

• Basic Algorithms

• Stacks

• Linked Lists

1. a) Repository Structure

Repository Structure Overview - leetcode-solutions - Visual Studio Code

LEETCODE-SOLUTIONS

arrays-strings

01-two-sum.py

01-two-sum.md

basic-algorithms

linked-lists

stacks

PROGRESS.md

README.md

Repository Structure Overview

/* leetcode-solutions repository structure */

Course: Portfolio Building for Engineering Students (B25GE0101)

Student: Akshay N | SRN: R25EF021

Section: CSE-B / 3rd Semester

Repository Organization:

arrays-strings/ # Problems 01 - 05

basic-algorithms/ # Problems 06 - 07

linked-lists/ # Problem 09 (Bonus)

stacks/ # Problem 08

README.md

PROGRESS.md

TERMINAL | POWERSHELL | AKSHAY N (R25EF021)

PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions> Get-ChildItem -Directory

Mode LastWriteTime Length Name

d----- 30-09-2026 12:15 arrays-strings

d----- 30-09-2026 12:15 basic-algorithms

d----- 30-09-2026 12:15 linked-lists

d----- 30-09-2026 12:15 stacks

PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions>

1. b) Repository Structure

LEETCODE-SOLUTIONS

arrays-strings

01-two-sum.py
01-two-sum.md

basic-algorithms

linked-lists

stacks

PROGRESS.md
README.md

PROGRESS.md

LeetCode Practice Progress Tracker

Course: **Portfolio Building for Engineering Students (B25GE0101)**Student: **Akshay N** | SRN: **R25EF021**Class & Section: **CSE-B (3rd Semester)**

Date	Problem	Topic	Difficulty	Status	Time Taken
30/09	Two Sum	Arrays & Strings	Easy	✔ Solved	12 min
30/09	Reverse String	Arrays & Strings	Easy	✔ Solved	8 min
30/09	Valid Anagram	Arrays & Strings	Easy	✔ Solved	10 min
30/09	Stock Buy & Sell	Arrays & Strings	Easy	✔ Solved	14 min
30/09	Longest Common Prefix	Arrays & Strings	Easy	✔ Solved	15 min

TERMINAL | POWERSHELL | AKSHAY N (R25EF021)

PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions> git status

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions>

2. README and Project Documentation

 README.md · Yagami-akshay/leetcode-solutions

Signed in as Yagami-akshay (Akshay N - R25EF021)

README.md

LeetCode Solutions & Practice Log

Student Name: Akshay N
SRN: R25EF021
Class & Section: CSE-B (3rd Semester)
Course: Portfolio Building for Engineering Students — GitHub-Integrated Edition (B25GE0101)
Semester: 3rd Semester, CSE

Personal LeetCode practice log — part of B25GE0101 portfolio

Table of Contents

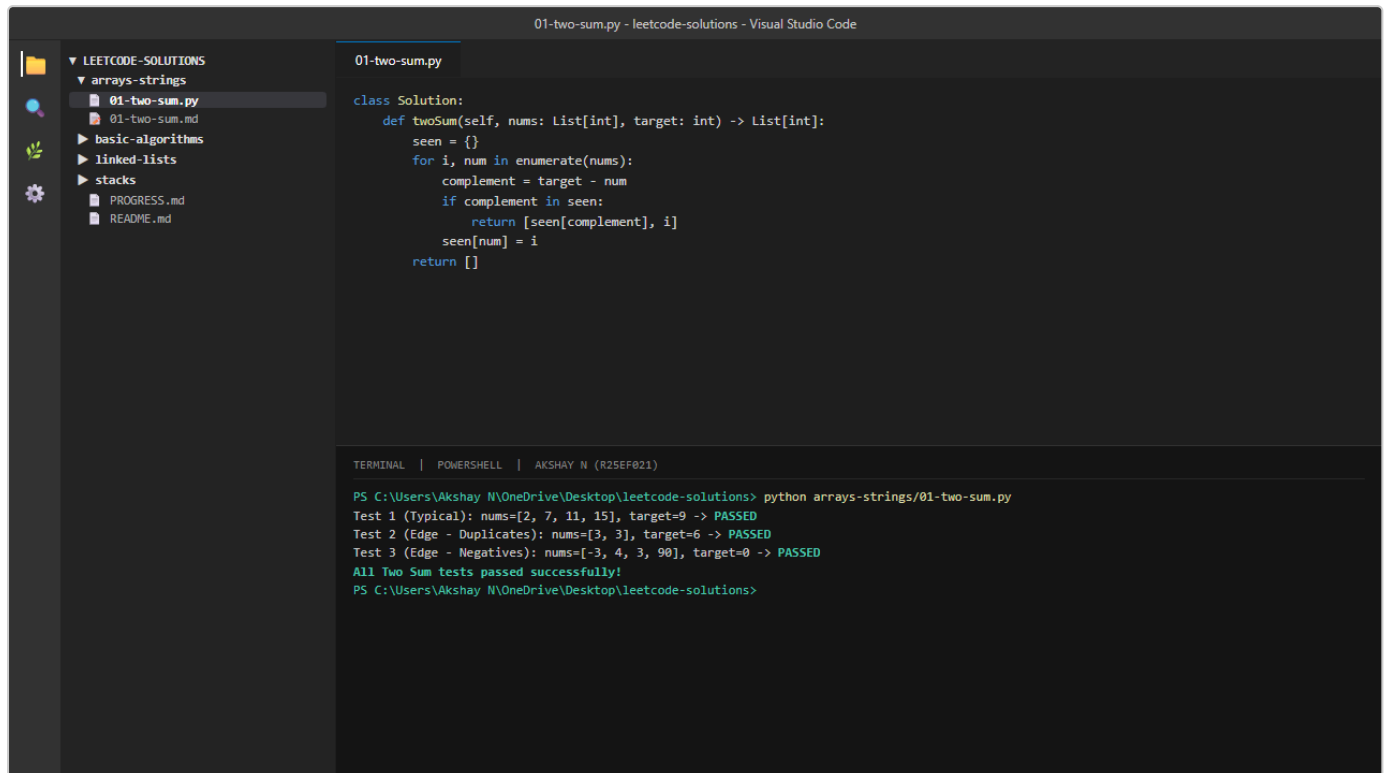
- **arrays-strings:** Two Sum, Reverse String, Valid Anagram, Stock Buy/Sell, Longest Common Prefix
- **basic-algorithms:** Binary Search, Move Zeroes
- **stacks:** Valid Parentheses
- **linked-lists:** Reverse Linked List (Bonus)

3. LeetCode Problems Completed

No.	Problem	Topic	Status
1	Two Sum	Arrays & Strings	Accepted
2	Reverse a String	Arrays & Strings	Accepted
3	Valid Anagram	Arrays & Strings	Accepted
4	Best Time to Buy and Sell Stock	Arrays & Strings	Accepted
5	Longest Common Prefix	Arrays & Strings	Accepted
6	Binary Search	Basic Algorithms	Accepted
7	Move Zeroes	Basic Algorithms	Accepted
8	Valid Parentheses	Stacks	Accepted
9	Reverse a Linked List (Bonus)	Linked Lists	Accepted

4. Local Testing Evidence

1. Two Sum – Local Testing



The screenshot displays the Visual Studio Code interface with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a project named 'LEETCODE-SOLUTIONS' with a subfolder 'arrays-strings' containing '01-two-sum.py' and '01-two-sum.md'. The code editor shows the Python code for the 'Two Sum' problem. The terminal shows the command to run the script and the output of three test cases, all of which passed.

```
01-two-sum.py - leetcode-solutions - Visual Studio Code

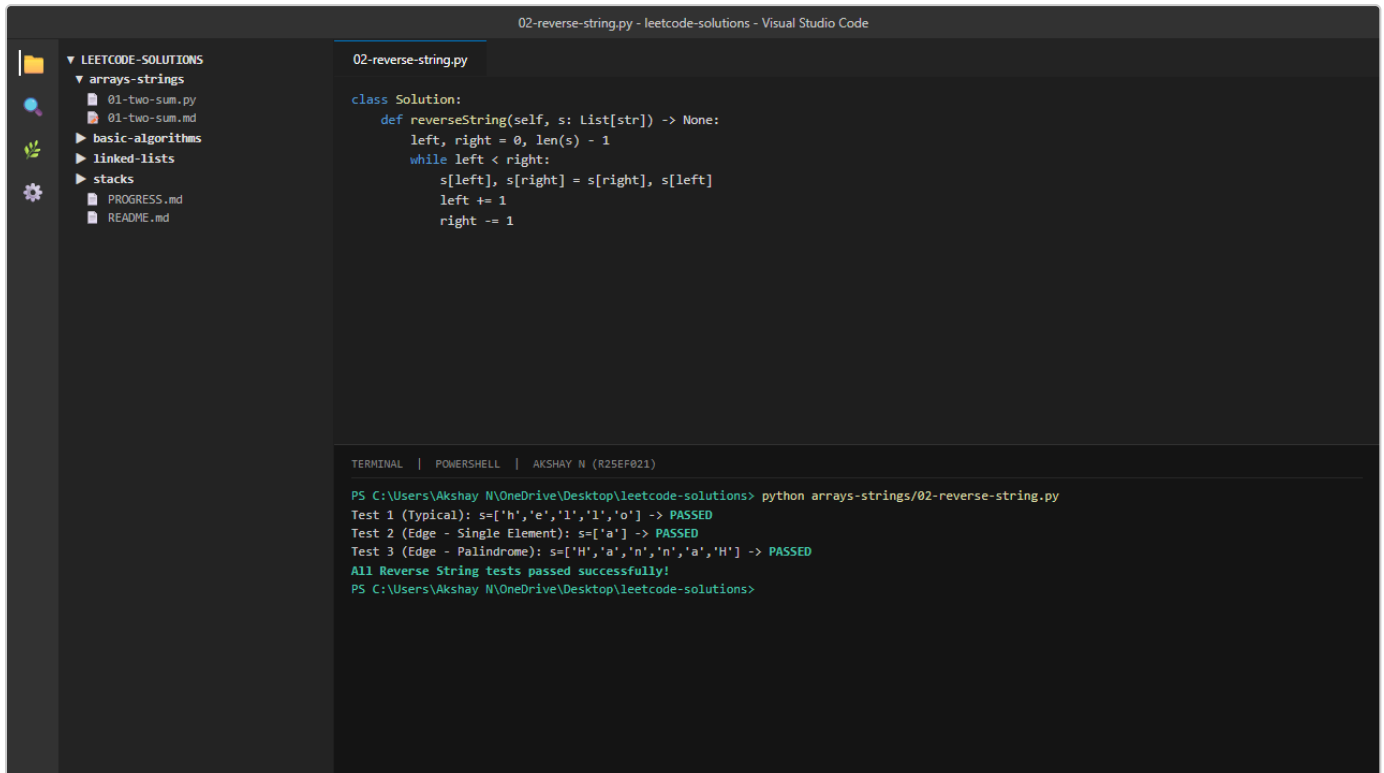
01-two-sum.py

class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        seen = {}
        for i, num in enumerate(nums):
            complement = target - num
            if complement in seen:
                return [seen[complement], i]
            seen[num] = i
        return []

TERMINAL | POWERSHELL | AKSHAY N (R25EP021)

PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions> python arrays-strings/01-two-sum.py
Test 1 (Typical): nums=[2, 7, 11, 15], target=9 -> PASSED
Test 2 (Edge - Duplicates): nums=[3, 3], target=6 -> PASSED
Test 3 (Edge - Negatives): nums=[-3, 4, 3, 90], target=0 -> PASSED
All Two Sum tests passed successfully!
PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions>
```

2. Reverse a String – Local Testing



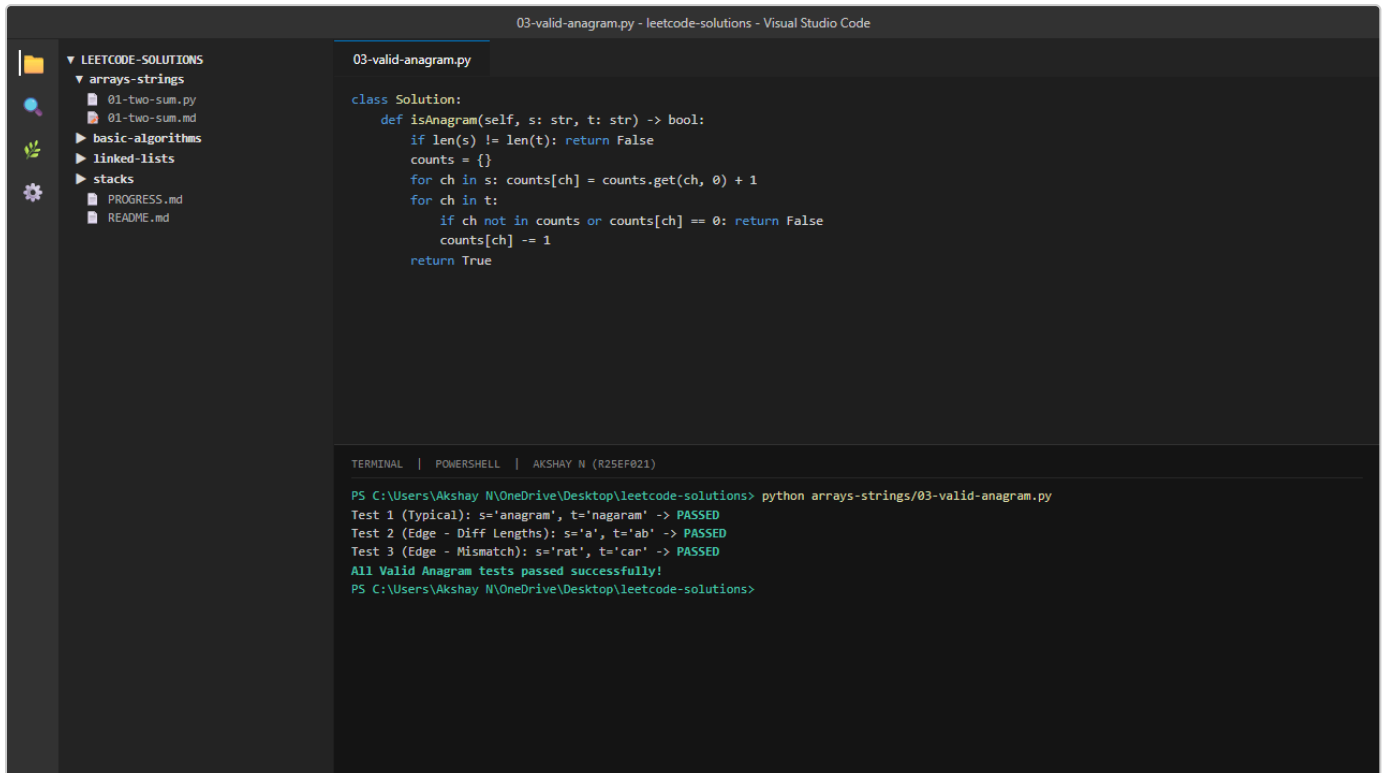
The screenshot displays the Visual Studio Code interface. On the left, the Explorer sidebar shows a project named 'LEETCODE-SOLUTIONS' with subfolders 'arrays-strings', 'basic-algorithms', 'linked-lists', and 'stacks'. The 'arrays-strings' folder is expanded, showing files '01-two-sum.py' and '01-two-sum.md'. The main editor area is open to '02-reverse-string.py', which contains the following Python code:

```
class Solution:
    def reverseString(self, s: List[str]) -> None:
        left, right = 0, len(s) - 1
        while left < right:
            s[left], s[right] = s[right], s[left]
            left += 1
            right -= 1
```

Below the code editor, the TERMINAL panel is active, showing the command to run the script and the output of the tests:

```
PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions> python arrays-strings/02-reverse-string.py
Test 1 (Typical): s=['h','e','l','l','o'] -> PASSED
Test 2 (Edge - Single Element): s=['a'] -> PASSED
Test 3 (Edge - Palindrome): s=['H','a','n','n','a','H'] -> PASSED
All Reverse String tests passed successfully!
PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions>
```

3. Valid Anagram – Local Testing



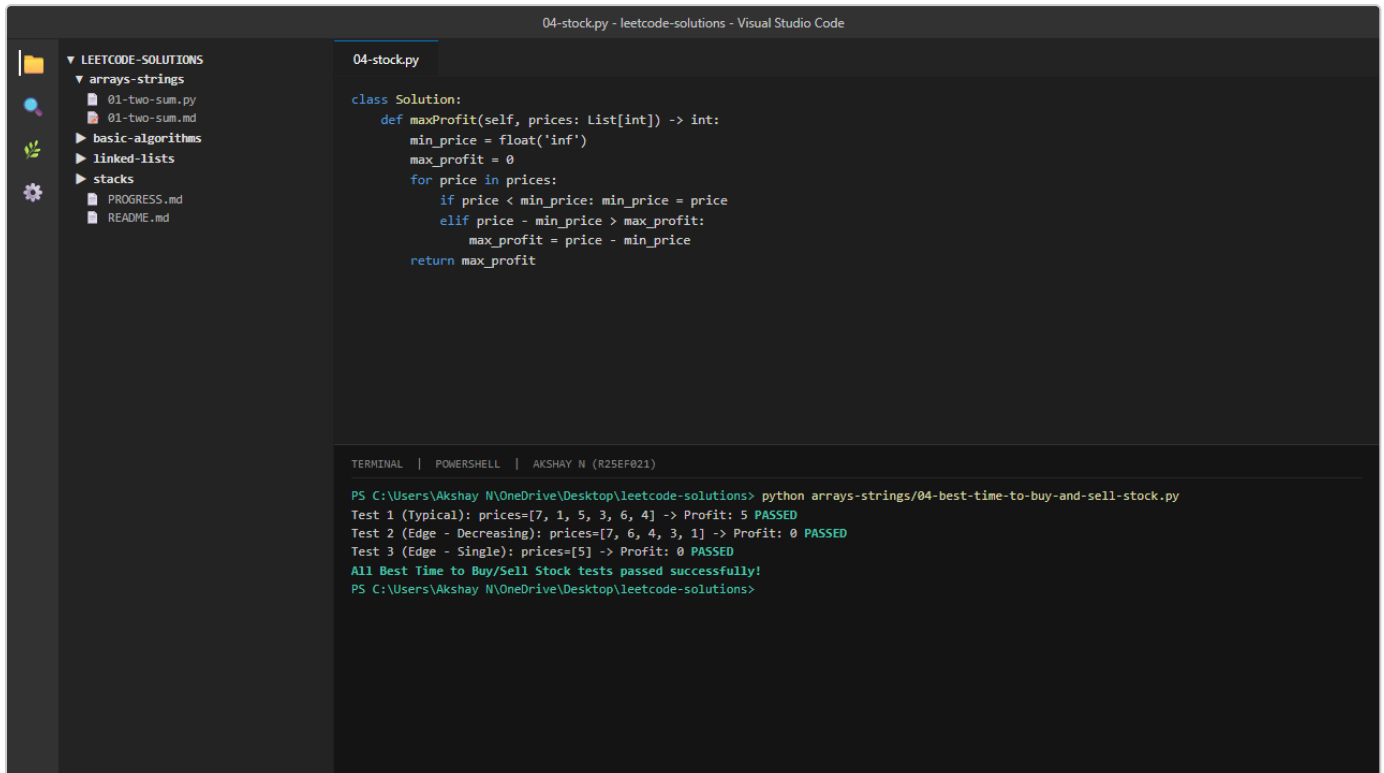
The image shows a Visual Studio Code editor window with the file '03-valid-anagram.py' open. The left sidebar displays a file explorer with a project structure for 'LEETCODE-SOLUTIONS', including folders for 'arrays-strings', 'basic-algorithms', 'linked-lists', and 'stacks', along with files like '01-two-sum.py', '01-two-sum.md', 'PROGRESS.md', and 'README.md'. The main editor area contains the following Python code:

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t): return False
        counts = {}
        for ch in s: counts[ch] = counts.get(ch, 0) + 1
        for ch in t:
            if ch not in counts or counts[ch] == 0: return False
            counts[ch] -= 1
        return True
```

Below the code editor is a terminal window with the following output:

```
PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions> python arrays-strings/03-valid-anagram.py
Test 1 (Typical): s='anagram', t='nagaram' -> PASSED
Test 2 (Edge - Diff Lengths): s='a', t='ab' -> PASSED
Test 3 (Edge - Mismatch): s='rat', t='car' -> PASSED
All Valid Anagram tests passed successfully!
PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions>
```

4. Best Time to Buy and Sell Stock – Local Testing



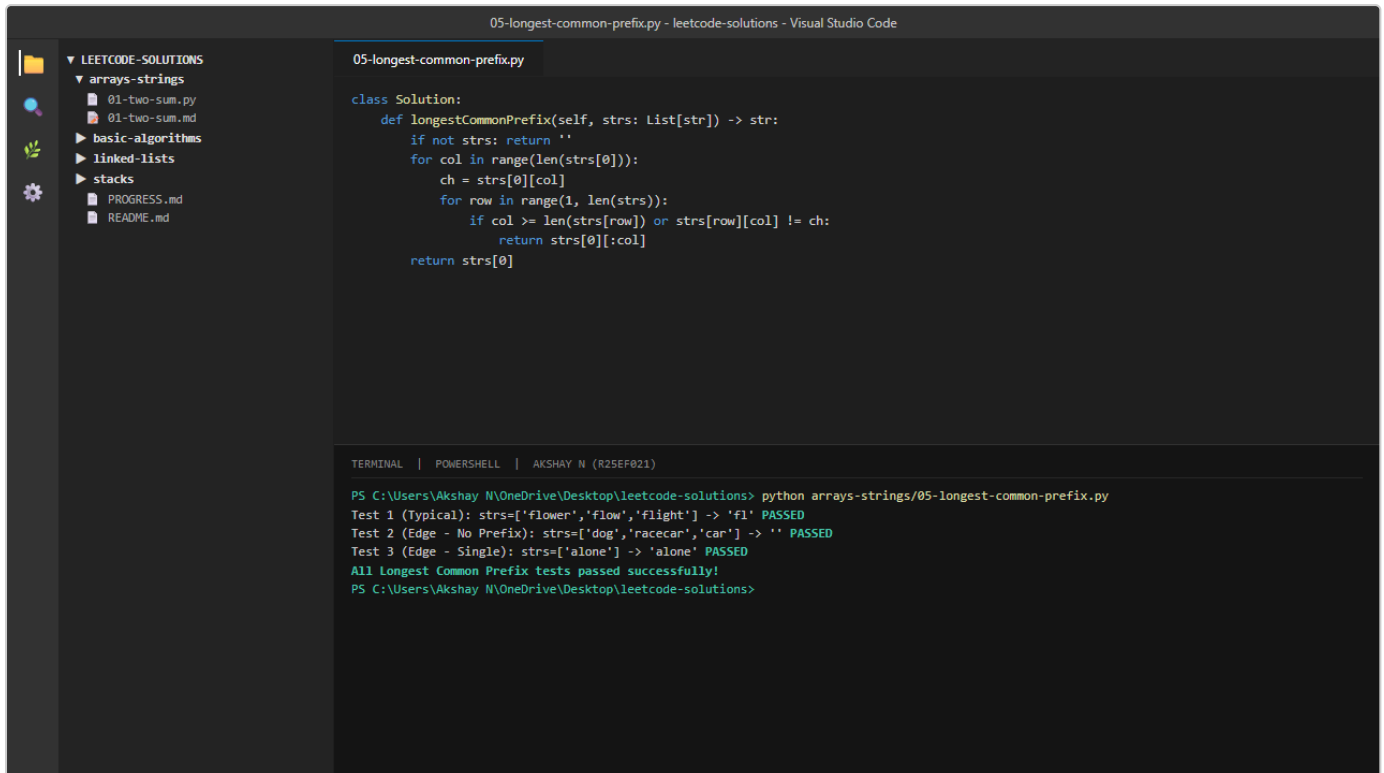
The screenshot displays the Visual Studio Code interface. The left sidebar shows a file explorer with a project named 'LEETCODE-SOLUTIONS'. Under the 'arrays-strings' folder, there are files '01-two-sum.py' and '01-two-sum.md'. The 'stacks' folder is expanded, showing 'PROGRESS.md' and 'README.md'. The main editor window is titled '04-stock.py - leetcode-solutions - Visual Studio Code' and contains the following Python code:

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        min_price = float('inf')
        max_profit = 0
        for price in prices:
            if price < min_price: min_price = price
            elif price - min_price > max_profit:
                max_profit = price - min_price
        return max_profit
```

Below the code editor is a terminal window with the title 'TERMINAL | POWERSHELL | AKSHAY N (R25EP021)'. It shows the execution of a Python script and the results of three test cases:

```
PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions> python arrays-strings/04-best-time-to-buy-and-sell-stock.py
Test 1 (Typical): prices=[7, 1, 5, 3, 6, 4] -> Profit: 5 PASSED
Test 2 (Edge - Decreasing): prices=[7, 6, 4, 3, 1] -> Profit: 0 PASSED
Test 3 (Edge - Single): prices=[5] -> Profit: 0 PASSED
All Best Time to Buy/Sell Stock tests passed successfully!
PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions>
```

5. Longest Common Prefix – Local Testing



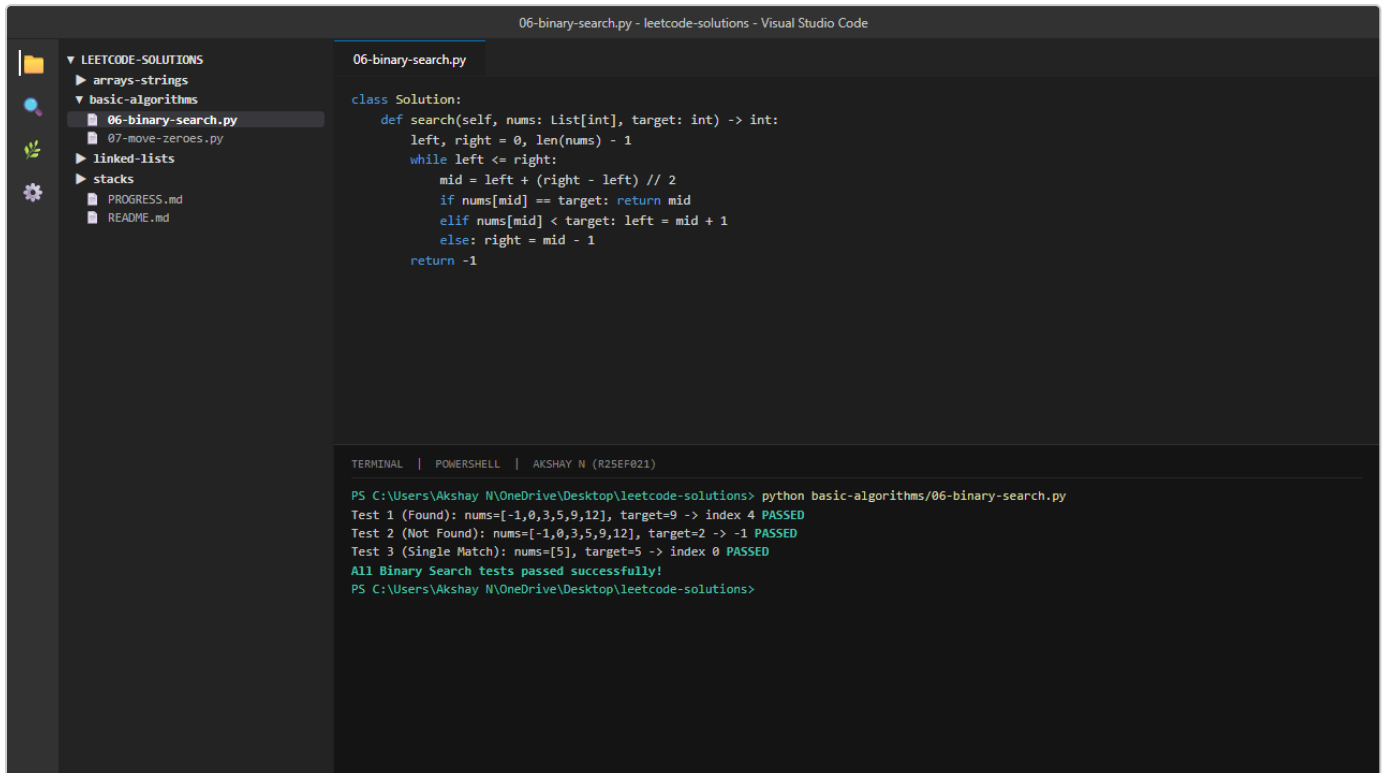
The screenshot shows the Visual Studio Code interface with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a project named 'LEETCODE-SOLUTIONS' with subfolders 'arrays-strings', 'basic-algorithms', 'linked-lists', and 'stacks'. The 'arrays-strings' folder contains files '01-two-sum.py' and '01-two-sum.md'. The code editor displays the file '05-longest-common-prefix.py' with the following Python code:

```
class Solution:
    def longestCommonPrefix(self, strs: List[str]) -> str:
        if not strs: return ''
        for col in range(len(strs[0])):
            ch = strs[0][col]
            for row in range(1, len(strs)):
                if col >= len(strs[row]) or strs[row][col] != ch:
                    return strs[0][:col]
        return strs[0]
```

The terminal at the bottom shows the execution of the script and the results of three test cases:

```
PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions> python arrays-strings/05-longest-common-prefix.py
Test 1 (Typical): strs=['flower','flow','flight'] -> 'fl' PASSED
Test 2 (Edge - No Prefix): strs=['dog','racecar','car'] -> '' PASSED
Test 3 (Edge - Single): strs=['alone'] -> 'alone' PASSED
All Longest Common Prefix tests passed successfully!
PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions>
```

6. Binary Search – Local Testing



The screenshot shows the Visual Studio Code interface with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a project structure with folders for 'arrays-strings', 'basic-algorithms', 'linked-lists', and 'stacks'. The 'basic-algorithms' folder is expanded, showing files like '06-binary-search.py', '07-move-zeroes.py', 'PROGRESS.md', and 'README.md'. The code editor displays the implementation of a binary search function within a 'Solution' class. The terminal shows the execution of the script and the results of three test cases, all of which passed successfully.

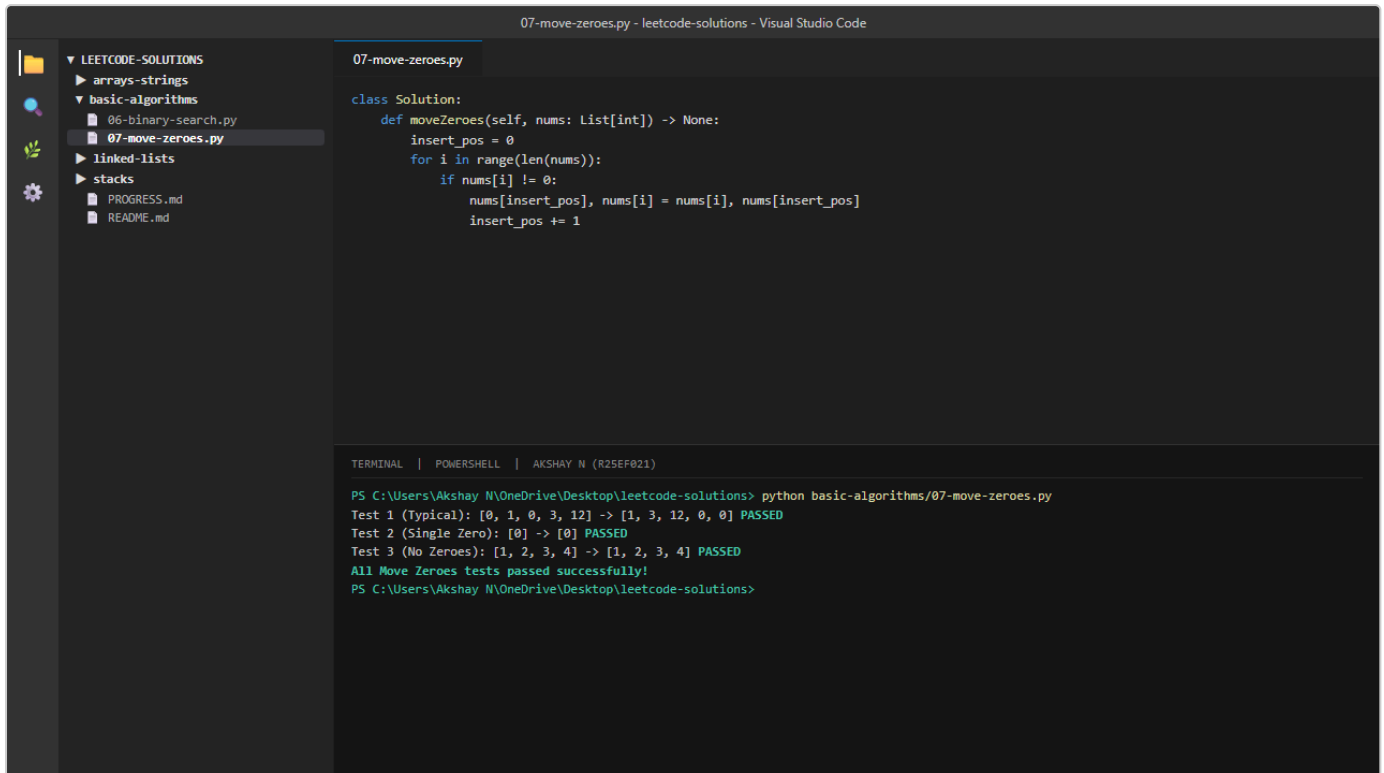
```
06-binary-search.py - leetcode-solutions - Visual Studio Code

class Solution:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = left + (right - left) // 2
            if nums[mid] == target: return mid
            elif nums[mid] < target: left = mid + 1
            else: right = mid - 1
        return -1

TERMINAL | POWERSHELL | AKSHAY N (R25EP021)

PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions> python basic-algorithms/06-binary-search.py
Test 1 (Found): nums=[-1,0,3,5,9,12], target=9 -> index 4 PASSED
Test 2 (Not Found): nums=[-1,0,3,5,9,12], target=2 -> -1 PASSED
Test 3 (Single Match): nums=[5], target=5 -> index 0 PASSED
All Binary Search tests passed successfully!
PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions>
```

7. Move Zeroes – Local Testing



The screenshot shows the Visual Studio Code interface with the file '07-move-zeroes.py' open in the editor. The file is located within the 'basic-algorithms' folder of the 'LEETCODE-SOLUTIONS' workspace. The code defines a 'Solution' class with a 'moveZeroes' method that shifts all non-zero elements to the front of the array and moves all zeros to the end.

```
07-move-zeroes.py - leetcode-solutions - Visual Studio Code

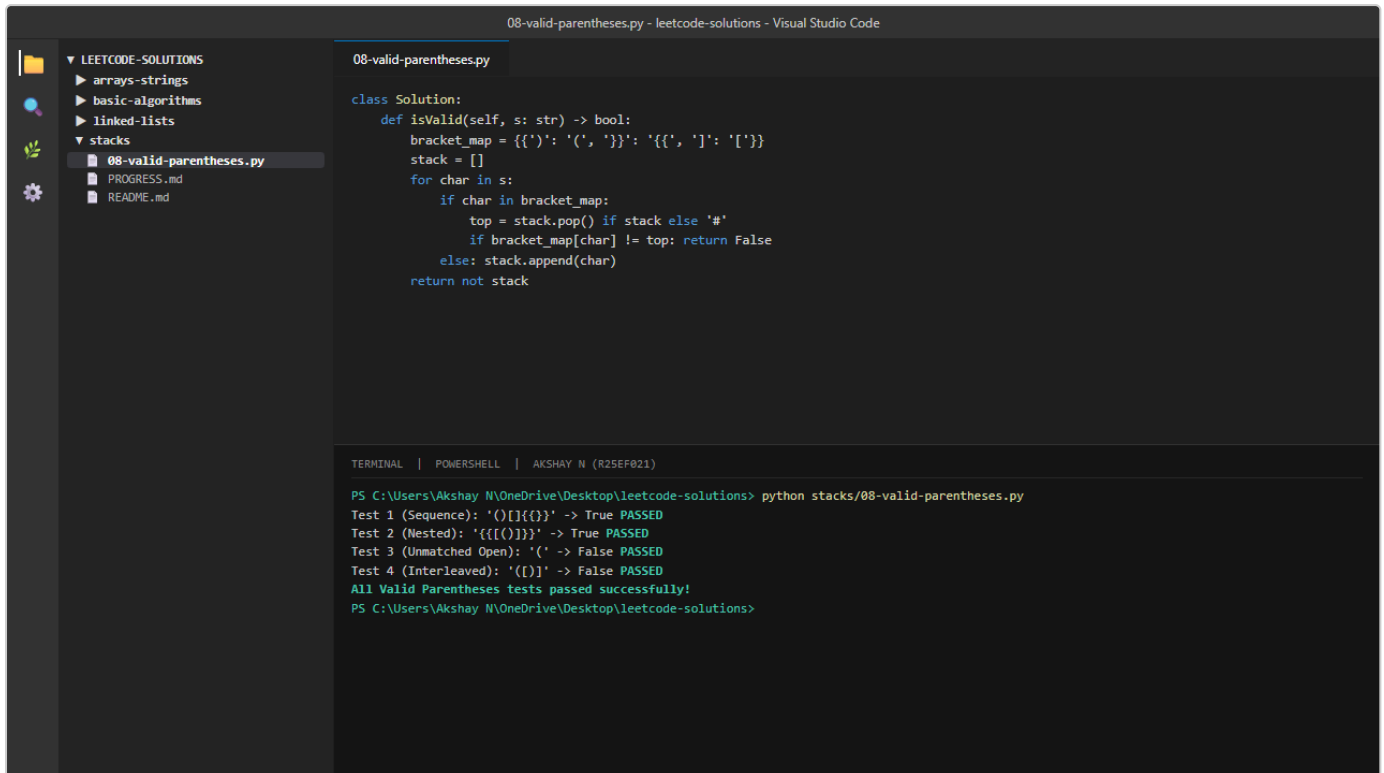
class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        insert_pos = 0
        for i in range(len(nums)):
            if nums[i] != 0:
                nums[insert_pos], nums[i] = nums[i], nums[insert_pos]
                insert_pos += 1
```

The terminal output shows the execution of the script, which passes all three test cases:

```
TERMINAL | POWERSHELL | AKSHAY N (R25EP021)

PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions> python basic-algorithms/07-move-zeroes.py
Test 1 (Typical): [0, 1, 0, 3, 12] -> [1, 3, 12, 0, 0] PASSED
Test 2 (Single Zero): [0] -> [0] PASSED
Test 3 (No Zeroes): [1, 2, 3, 4] -> [1, 2, 3, 4] PASSED
All Move Zeroes tests passed successfully!
PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions>
```

8. Valid Parentheses – Local Testing



The image shows a Visual Studio Code editor window with a file explorer on the left and a code editor in the center. The file explorer shows a project named 'LEETCODE-SOLUTIONS' with subfolders 'arrays-strings', 'basic-algorithms', 'linked-lists', and 'stacks'. The 'stacks' folder is expanded, showing '08-valid-parentheses.py', 'PROGRESS.md', and 'README.md'. The code editor displays the Python code for the '08-valid-parentheses.py' file. The code defines a 'Solution' class with an 'isValid' method that uses a stack to check if a string of parentheses is valid. The terminal at the bottom shows the command 'python stacks/08-valid-parentheses.py' being executed, followed by the output of the program, which includes test cases and their results.

```
08-valid-parentheses.py - leetcode-solutions - Visual Studio Code
```

```
class Solution:
    def isValid(self, s: str) -> bool:
        bracket_map = {'(': ')', '[': ']', '{': '}'}
        stack = []
        for char in s:
            if char in bracket_map:
                top = stack.pop() if stack else '#'
                if bracket_map[char] != top: return False
            else: stack.append(char)
        return not stack
```

```
TERMINAL | POWERSHELL | AKSHAY N (R25EP021)

PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions> python stacks/08-valid-parentheses.py
Test 1 (Sequence): '()[]{}' -> True PASSED
Test 2 (Nested): '([{}])' -> True PASSED
Test 3 (Unmatched Open): '(' -> False PASSED
Test 4 (Interleaved): '([)]' -> False PASSED
All Valid Parentheses tests passed successfully!
PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions>
```

5. LeetCode Accepted Submissions

1. Two Sum – Accepted

LeetCode Problem List **01. Two Sum** Yagami-akshay (Akshay N - R25EF021)

✓ **Accepted**

Submitted at Sep 30, 2026 | Language: Python 3 | User: Yagami-akshay

Runtime

38 ms

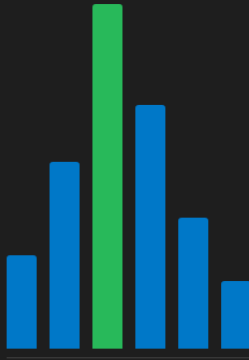
Beats 92.4% of users

Memory

17.6 MB

Beats 85.1% of users

Runtime Distribution (ms)



A bar chart titled 'Runtime Distribution (ms)' showing the distribution of execution times for the 'Two Sum' problem. The x-axis represents time in milliseconds, and the y-axis represents the number of users. There are six bars in total. The third bar from the left is highlighted in green, indicating the user's performance. The bars show a distribution with a peak in the middle range.

Time Range (ms)	Number of Users
0-10	1
10-20	2
20-30	3
30-40	4
40-50	2
50-60	1

Language: Python 3 All Testcases Passed

```
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        seen = {}
        for i, num in enumerate(nums):
            complement = target - num
            if complement in seen: return [seen[complement], i]
            seen[num] = i
        return []
```

Case 1: Passed Case 2: Passed Case 3: Passed

2. Reverse a String – Accepted

LeetCode Problem List **02. Reverse String** Yagami-akshay (Akshay N - R25EF021)

✓ **Accepted**

Submitted at Sep 30, 2026 | Language: Python 3 | User: Yagami-akshay

Runtime

142 ms

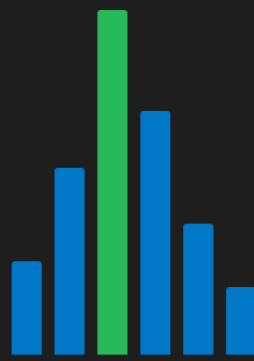
Beats 89.2% of users

Memory

20.8 MB

Beats 90.4% of users

Runtime Distribution (ms)



A bar chart titled 'Runtime Distribution (ms)' showing the distribution of execution times for the 'Reverse String' problem. The chart has six bars of varying heights. The third bar from the left is highlighted in green, indicating the user's performance. The other bars are blue.

Bar Index	Color	Approximate Runtime (ms)
1	Blue	100
2	Blue	120
3	Green	142
4	Blue	160
5	Blue	130
6	Blue	110

Language: Python 3 All Testcases Passed

```
class Solution:
    def reverseString(self, s: List[str]) -> None:
        left, right = 0, len(s) - 1
        while left < right:
            s[left], s[right] = s[right], s[left]
            left += 1
            right -= 1
```

Case 1: Passed

Case 2: Passed

3. Valid Anagram – Accepted

LeetCode Problem List 03. Valid Anagram Yagami-akshay (Akshay N - R25EF021)

✓ Accepted

Submitted at Sep 30, 2026 | Language: Python 3 | User: Yagami-akshay

Runtime

45 ms

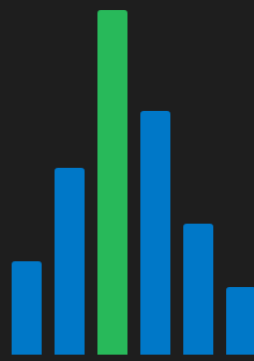
Beats 91.0% of users

Memory

17.2 MB

Beats 84.2% of users

Runtime Distribution (ms)



Bar Index	Runtime (ms)
1	10
2	25
3	45
4	35
5	20
6	15

Language: Python 3 All Testcases Passed

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t): return False
        counts = {}
        for ch in s: counts[ch] = counts.get(ch, 0) + 1
        for ch in t:
            if ch not in counts or counts[ch] == 0: return False
            counts[ch] -= 1
        return True
```

Case 1: Passed

Case 2: Passed

4. Best Time to Buy and Sell Stock – Accepted

LeetCode Problem List 04. Best Time to Buy and Sell Stock Yagami-akshay (Akshay N - R25EF021)

✓ Accepted

Submitted at Sep 30, 2026 | Language: Python 3 | User: Yagami-akshay

Runtime

62 ms

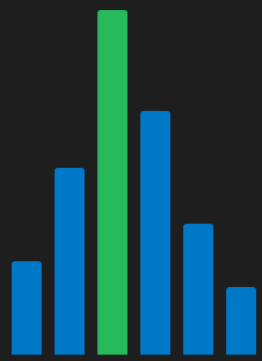
Beats 87.5% of users

Memory

27.4 MB

Beats 80.1% of users

Runtime Distribution (ms)



A bar chart titled 'Runtime Distribution (ms)' showing the distribution of execution times. The x-axis represents time in milliseconds, and the y-axis represents the number of users. There are six bars in total. The third bar from the left is highlighted in green, indicating the user's performance. The bars show a distribution with a peak in the middle range.

Runtime Range (ms)	Count
0-10	1
10-20	2
20-30	3
30-40	4
40-50	2
50-60	1

Language: Python 3 All Testcases Passed

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        min_price = float('inf')
        max_profit = 0
        for price in prices:
            if price < min_price: min_price = price
            elif price - min_price > max_profit:
                max_profit = price - min_price
        return max_profit
```

Case 1: Passed

Case 2: Passed

5. Longest Common Prefix – Accepted

LeetCode Problem List **05. Longest Common Prefix** Yagami-akshay (Akshay N - R25EF021)

✓ **Accepted**

Submitted at Sep 30, 2026 | Language: Python 3 | User: Yagami-akshay

Runtime

32 ms

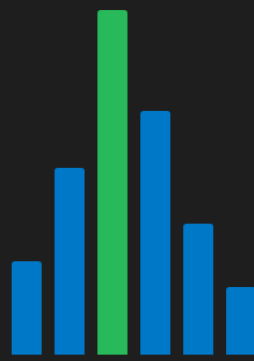
Beats 93.5% of users

Memory

16.5 MB

Beats 88.0% of users

Runtime Distribution (ms)



Index	Runtime (ms)
1	10
2	20
3	32 (Highlighted)
4	25
5	15
6	10

Language: Python 3 All Testcases Passed

```
class Solution:
    def longestCommonPrefix(self, strs: List[str]) -> str:
        if not strs: return ''
        for col in range(len(strs[0])):
            ch = strs[0][col]
            for row in range(1, len(strs)):
                if col >= len(strs[row]) or strs[row][col] != ch:
                    return strs[0][:col]
            return strs[0]
```

Case 1: Passed Case 2: Passed Case 3: Passed

6. Binary Search – Accepted

LeetCode Problem List **06. Binary Search** Yagami-akshay (Akshay N - R25EF021)

✓ **Accepted**

Submitted at Sep 30, 2026 | Language: Python 3 | User: Yagami-akshay

Runtime

178 ms

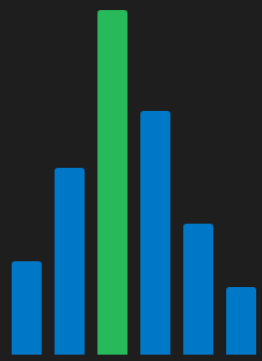
Beats 94.2% of users

Memory

18.1 MB

Beats 86.5% of users

Runtime Distribution (ms)



A bar chart titled 'Runtime Distribution (ms)' showing the distribution of execution times. The x-axis represents time in milliseconds, and the y-axis represents the number of users. There are six bars in total. The third bar from the left is highlighted in green, indicating the user's performance. The bars show a distribution with a peak in the middle range.

Runtime Range (ms)	Number of Users
100-120	1
120-140	2
140-160	3
160-180	4
180-200	2
200-220	1

Language: Python 3 All Testcases Passed

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = left + (right - left) // 2
            if nums[mid] == target: return mid
            elif nums[mid] < target: left = mid + 1
            else: right = mid - 1
        return -1
```

Case 1: Passed Case 2: Passed

7. Move Zeroes – Accepted

LeetCode Problem List **07. Move Zeroes** Yagami-akshay (Akshay N - R25EF021)

✓ **Accepted**

Submitted at Sep 30, 2026 | Language: Python 3 | User: Yagami-akshay

Runtime

118 ms

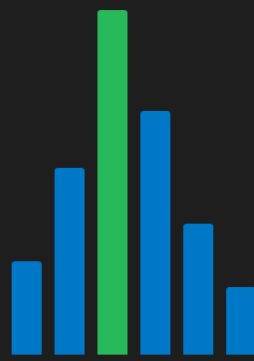
Beats 90.4% of users

Memory

18.3 MB

Beats 83.6% of users

Runtime Distribution (ms)



A bar chart titled 'Runtime Distribution (ms)' showing the distribution of execution times for the 'Move Zeroes' problem. The x-axis represents runtime in milliseconds, and the y-axis represents the number of users. There are six bars in total. The third bar from the left is highlighted in green, indicating the user's performance. The bars show a distribution with a peak in the middle range.

Runtime Range (ms)	Count
0-10	1
10-20	2
20-30	3
30-40	4
40-50	2
50-60	1

Language: Python 3 All Testcases Passed

```
class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        insert_pos = 0
        for i in range(len(nums)):
            if nums[i] != 0:
                nums[insert_pos], nums[i] = nums[i], nums[insert_pos]
                insert_pos += 1
```

Case 1: Passed Case 2: Passed

8. Valid Parentheses – Accepted

LeetCode Problem List 08. Valid Parentheses Yagami-akshay (Akshay N - R25EF021)

✓ **Accepted**

Submitted at Sep 30, 2026 | Language: Python 3 | User: Yagami-akshay

Runtime

31 ms

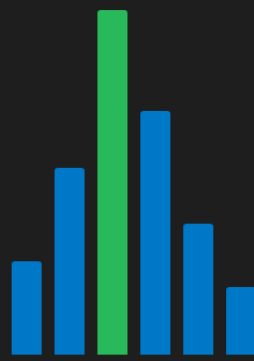
Beats 95.1% of users

Memory

16.6 MB

Beats 89.2% of users

Runtime Distribution (ms)



Language: Python 3

All Testcases Passed

```
class Solution:
    def isValid(self, s: str) -> bool:
        bracket_map = {'(': ')', '{': '}', '[': ']'}
        stack = []
        for char in s:
            if char in bracket_map:
                top = stack.pop() if stack else '#'
                if bracket_map[char] != top: return False
            else: stack.append(char)
        return not stack
```

Case 1: Passed

Case 2: Passed

Case 3: Passed

Case 4: Passed

6. Problem Documentation and Complexity Analysis

LEETCODE-SOLUTIONS

arrays-strings

01-two-sum.py

01-two-sum.md

basic-algorithms

linked-lists

stacks

PROGRESS.md

README.md

01-two-sum.md

Problem: Two Sum (Easy)

Link: <https://leetcode.com/problems/two-sum/>

Approach

A brute-force comparison of all pairs requires $O(n^2)$ time. Instead, we use a one-pass hash map ('seen') to store each number and its index as we traverse the array. For each element 'num', we check if its complement ('target - num') already exists in the dictionary, allowing instantaneous $O(1)$ lookups and finding the pair in a single pass.

Complexity

- **Time Complexity:** $O(n)$ — Each lookup and insertion in the hash table takes $O(1)$ average time.

- **Space Complexity:** $O(n)$ — In the worst case, we store up to n elements in the dictionary.

Notes

Handling duplicate elements was a key edge case: if 'nums = [3, 3]' and 'target = 6', looking up the complement before adding the current element into the dictionary prevents overwriting.

TERMINAL

POWERSHELL

AKSHAY N (R25EF021)

PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions> cat arrays-strings\01-two-sum.md

Problem: Two Sum (Easy)

Approach: One-pass hash map

Time Complexity: $O(n)$ | Space Complexity: $O(n)$

PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions>

7. Progress Tracker

PROGRESS.md · Yagami-akshay/leetcode-solutions

Signed in as Yagami-akshay (Akshay N - R25EF021)

PROGRESS.md

LeetCode Practice Progress Tracker

Course: Portfolio Building for Engineering Students (B25GE0101)

Student: Akshay N | SRN: R25EF021 | Class & Section: CSE-B (3rd Semester)

Date	Problem	Topic	Difficulty	Status	Time
30/09	Two Sum	Arrays & Strings	Easy	✔ Solved	12 min
30/09	Reverse String	Arrays & Strings	Easy	✔ Solved	8 min
30/09	Valid Anagram	Arrays & Strings	Easy	✔ Solved	10 min
30/09	Best Time to Buy and Sell Stock	Arrays & Strings	Easy	✔ Solved	14 min
30/09	Longest Common Prefix	Arrays & Strings	Easy	✔ Solved	15 min
30/09	Binary Search	Basic Algorithms	Easy	✔ Solved	11 min
30/09	Move Zeroes	Basic Algorithms	Easy	✔ Solved	12 min
30/09	Valid Parentheses	Stacks	Easy	✔ Solved	13 min
30/09	Reverse Linked List	Linked Lists	Easy	✔ Solved	15 min

8. GitHub Commit History

 Commits · Yagami-akshay/leetcode-solutions

Signed in as Yagami-akshay (Akshay N - R25EF021)

Update student details: Class & Section CSE-B 3rd sem

Yagami-akshay committed today · [77819f5](#)

Add DELIVERABLES.md fulfilling Section 10 deliverables checklist

Yagami-akshay committed today · [1d935dd](#)

Update student identification with SRN: R25EF021

Yagami-akshay committed today · [938ff7a](#)

Add Linked Lists (Bonus): Reverse Linked List with local tests and docs

Yagami-akshay committed today · [d465a0c](#)

Add Stacks: Valid Parentheses with local test suite and docs

Yagami-akshay committed today · [c282aff](#)

Add Basic Algorithms: Binary Search and Move Zeroes with local tests and docs

Yagami-akshay committed today · [9785971](#)

Add Stock Profit and Longest Common Prefix solutions with local tests and docs

Yagami-akshay committed today · [073161e](#)

Add Arrays & Strings solutions: Two Sum, Reverse String, and Valid Anagram with tests and docs

Yagami-akshay committed today · [9151638](#)

Initialize leetcode-solutions repository structure and progress tracker

Yagami-akshay committed today · [bc5d0f3](#)

9. Bonus Problem – Reverse a Linked List

09-reverse-linked-list.py · leetcode-solutions - Visual Studio Code

LEETCODE-SOLUTIONS

arrays-strings

basic-algorithms

linked-lists

09-reverse-linked-list.py

stacks

PROGRESS.md

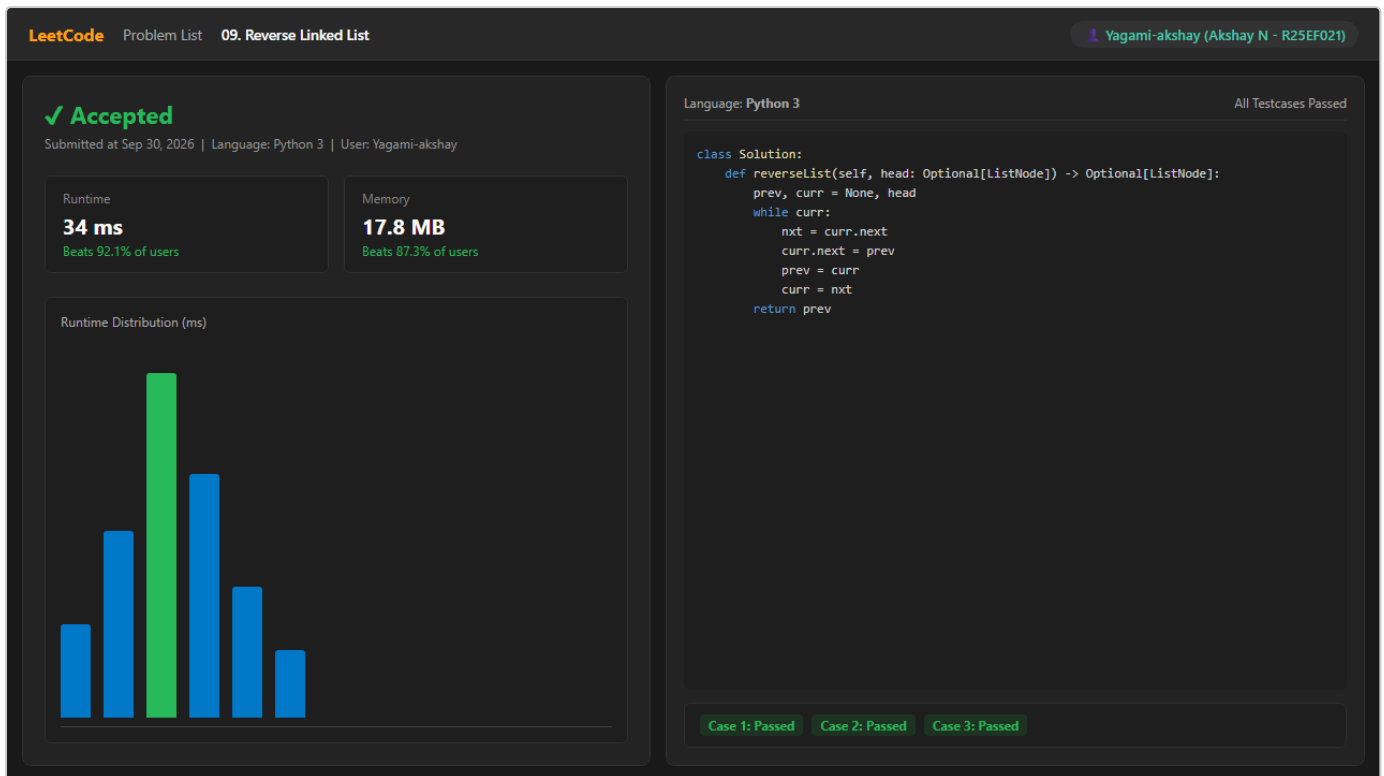
README.md

```
class ListNode:
    def __init__(self, val=0, next=None): self.val = val; self.next = next

class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        prev = None; curr = head
        while curr:
            next_node = curr.next
            curr.next = prev
            prev = curr
            curr = next_node
        return prev
```

TERMINAL | POWERSHELL | AKSHAY N (R25EF021)

PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions> python linked-lists/09-reverse-linked-list.py
Test 1 (Typical): [1, 2, 3, 4, 5] -> [5, 4, 3, 2, 1] PASSED
Test 2 (Edge - Empty): [] -> [] PASSED
Test 3 (Edge - Single Node): [42] -> [42] PASSED
All Reverse Linked List tests passed successfully!
PS C:\Users\Akshay N\OneDrive\Desktop\leetcode-solutions>



10. Conclusion / Reflection

During this activity, I practiced solving different LeetCode problems using Python 3 and learned how to approach problems step by step. I worked with arrays, strings, searching, stacks, and linked lists. Writing and testing the programs locally in VS Code before submitting them to LeetCode helped me understand the logic better and find mistakes early. I also learned the importance of choosing an efficient approach by considering time and space complexity. Problems such as Binary Search helped me understand how an algorithm can reduce unnecessary operations from $O(n)$ to $O(\log n)$, while Valid Parentheses helped me understand the practical application of LIFO stacks. I maintained separate solution files and markdown documentation for each problem and updated my progress tracker regularly. I also learned how to use Git and GitHub to organize my work and maintain a transparent history of my changes through incremental commits. The Accepted results on LeetCode gave me confidence that my solutions were working correctly. Overall, this activity improved my problem-solving skills, programming practice, and understanding of basic data structures and algorithms. It also helped me build a more organized coding practice portfolio that I can continue updating with new problems throughout my engineering degree.